mini-MQ: A Learning-Scale Distributed Message Queue with Chaos-Verified Durability and Honest Benchmarks

Jianhua Guo
Shenzhen University
1125448766@qq.com

Abstract

Log-based message queues such as Kafka, RocketMQ, and Pulsar are critical infrastructure, yet their size makes their load-bearing mechanisms hard to study as executable systems. We present **mini-MQ**, a distributed message queue of $\sim$16.7k lines of Java that reproduces those mechanisms with industrial fidelity: a RocketMQ-style single CommitLog with sparse consume indexes and CRC-based crash recovery; a 16-byte framed binary protocol; a two-parameter group-commit flush that raises forced-fsync throughput from 83 to 714,285 msg/s; Kafka-style pull replication with leader-epoch arbitration and high watermarks; and a Raft-style election kernel with PreVote. Durability is not asserted but *chaos-verified*: a 12-cell fault matrix with deterministic injection hooks kills, partitions, and resurrects leaders under load and hard-asserts RPO $= 0$ per cell under `acks=all`, with worst-case recovery (RTO) of 2.9 s. We benchmark honestly against Kafka 4.1.2 on the same host—51% of its throughput and 1.65$\times$ its P99 latency—and attribute the gap to four profiled mechanisms, explicitly declaring the share we cannot yet explain. Every number in this paper regenerates from committed evidence files by script.

1 Introduction

Distributed message queues are the connective tissue of modern data infrastructure: they decouple producers from consumers, absorb load spikes, and act as the durable commit log around which streaming architectures are built [10, 11]. Their designs encode decades of hard-won mechanism knowledge—sequential-write storage layouts, pull-based replication, quorum-managed in-sync replica sets, group-commit fsync amortization, zero-copy delivery. Yet precisely because these systems are production hardened, they are difficult to learn from directly: the mechanisms sit buried inside hundreds of thousands of lines of mature code, and textbook descriptions diverge from the code that actually runs.

This paper describes **mini-MQ**, a message queue built to make those mechanisms *executable* at a scale a single engineer can hold in their head. The goal is not to compete with Kafka on features; the goal is industrial *fidelity of the core* with pedagogical *size*: five core modules totaling $\sim$16.7k lines of Java (storage 3.4k, protocol 0.7k, replication 2.7k, broker 4.2k, client 5.6k), plus a 2.5k-line benchmark harness. Three commitments distinguish mini-MQ from a typical course project:

Real processes, real faults. A cluster is three real JVMs. Leaders are killed with `SIGKILL` mid-write-storm; the network is split bidirectionally and unidirectionally with a TCP proxy shield; a stale leader is resurrected to fight for its old position. Every failure cell ends with a hard reconciliation: *acked messages read back exactly, or the test fails*.

Deterministic chaos. Faults fire at *named program moments*— during an election, inside a flush window, inside a write storm—using atomic-counter time-point hooks, so rerunning one command reproduces the same fault at the same moment (§4).

Honest measurement. Every claim in this paper traces to a committed evidence file, and the comparison against Kafka reports the numbers we *lost*: 51% of Kafka's throughput and 1.65$\times$ its P99, with the gap attributed bucket-by-bucket to profiled mechanisms and the unexplained remainder explicitly declared rather than hidden (§5.4).

This last commitment deserves a word of motivation. Small-system writeups tend to report green dashboards: tests pass, the demo works. We instead treat *measurement honesty* as a first-class artifact, on the argument that a learning-scale system's credibility comes from verified invariants (what is asserted, under injected faults, per cell) and from calibrated expectations (what the system demonstrably is, relative to the industry reference on the same hardware). Concretely, mini-MQ's reliability claims are backed by a 12-cell chaos matrix with per-cell RPO assertions, and its performance claims are backed by same-host Kafka runs whose parameters were

aligned along seven dimensions—with every asymmetry that remained (round length, client batching) declared in the results table.

Our contributions:

- **A hybrid architecture at learning scale** (§3): RocketMQ's storage gene (single CommitLog, sparse ConsumeQueue), Kafka's replication gene (pull-based fetch, leader epochs, high watermarks, ISR with `min.insync.replicas` = 2 loud failure), and a Raft-style election kernel (terms, majority, PreVote)—the same control-plane/data-plane split that Kafka itself made in KRaft [18].
- **Chaos-verified durability** (§4): a deterministic fault-injection toolkit and a 12-cell matrix asserting RPO = 0 under `acks=all` for leader/follower `kill -9` during elections, flush windows, and write storms; bidirectional and one-way partitions; stale-leader revival; ISR shrink; minority partition; and retry storms. Worst-case RTO is 2.9 s against an 8 s budget; the two *loud* unavailability windows (18.3–23.7 s and 4.4–4.7 s) are reported separately rather than folded into RTO. The matrix found two real bugs before it turned green (§4.4).
- **Mechanism-level performance with profiled evidence** (§5): two-parameter group commit turning a 38,055× fsync wall into an 11% cost at 8 producers (records per fsync rising 241→909); profiling-driven optimization of the send-path lock critical section (2.4%→0.3% of samples, +3.4% throughput) and of hot-path allocations (stack-rental object pools, −17.3% sampled bytes); and a replication-loop race fix that took false ISR evictions from 5,308 per 60,000 writes to zero.
- **An honest same-host comparison with Kafka 4.1.2** (§5.4): 38,001 vs. 74,776 msg/s end-to-end (RF=3, `acks=all`, 1 KiB), P99 1,555 vs. 943 ms, with P50 parity (711 vs. 725 ms)—attributed to four buckets with profiled evidence, and a declared unattributed remainder.

mini-MQ is fully open source; one script builds it, boots a three-node cluster, reconciles a 10,000-message smoke run, and tears it down in under ten minutes (§8).

2 Background and Design Lineage

2.1 The log as the primitive

An append-only, totally ordered log is the simplest durable structure that supports both queueing and replay [11]. The engineering questions are all in the margins: how to write it fast (sequential writes, page cache, batching of `fsync`), how to replicate it safely (quorums, epochs, watermarks), and how to find data in it afterwards (indexes). Production systems answer these differently, and their answers are the "genes" a learning-scale system must choose among.

2.2 Three production genes

Kafka [5, 12] keeps one partitioned log per topic-partition and replicates by *pull*: followers fetch byte ranges from the leader, which fences stale leaders with *leader epochs* and publishes the committed prefix as a *high watermark* (HW). Replication membership is the ISR; writes with `acks=all` wait for the full ISR. Since KRaft [18], Kafka separates a Raft-based metadata quorum (control plane) from the per-partition fetch data plane. **RocketMQ** [7] instead appends *all* topics to a single per-broker CommitLog and builds sparse per-topic ConsumeQueue indexes by replay, buying one sequential write stream at the cost of read indirection. **Pulsar** [6] segments every topic into ledgers spread over BookKeeper bookies, decoupling storage from compute.

mini-MQ takes RocketMQ's storage gene (a single CommitLog per broker is the cleanest sequential-write layout at three-node scale), Kafka's replication gene (pull fetch, epochs, HW, ISR), and a Raft-style election kernel for the control plane—the same hybrid Kafka arrived at with KRaft [16, 18]. Eight architecture decision records, each with rejected alternatives and explicit invalidation boundaries, document why [15]; e.g., layered storage was rejected because a three-node deployment cannot cash its benefits.

2.3 The economics of durability

`fsync` on a spinning or NVMe-backed filesystem serializes at roughly one call per millisecond; a naive *ack-after-every-fsync* policy caps throughput at the fsync rate. Group commit—accumulate concurrent writes, flush once, acknowledge all—is the classic answer in databases [9], shipped as `ordered_commit()` in MySQL's binlog [3]. §5.2 measures this wall in mini-MQ (83 msg/s per-record force) and the group-commit curve that removes it.

3 Architecture

Figure 1 shows the system. Five modules implement it; we summarize each mechanism and its evidence anchor.

3.1 Storage engine

All messages from all topics append to one per-broker **CommitLog** of 256 MB mmap-backed segments [13]. Each record carries a fixed header, the payload, and a CRC; **crash recovery** is a full CRC scan that truncates a partial tail record, after which the sparse per-topic **ConsumeQueue** indexes are rebuilt by replaying the log. Byte arithmetic is `long` end-to-end—a discipline bought the hard way (§5.6 documents the 2 GiB overflow that a 30-minute soak exposed). Consume tail latency benefits from the page cache: back-to-back tail consumption reaches 160,498 msg/s (P99 120 ms, message-age

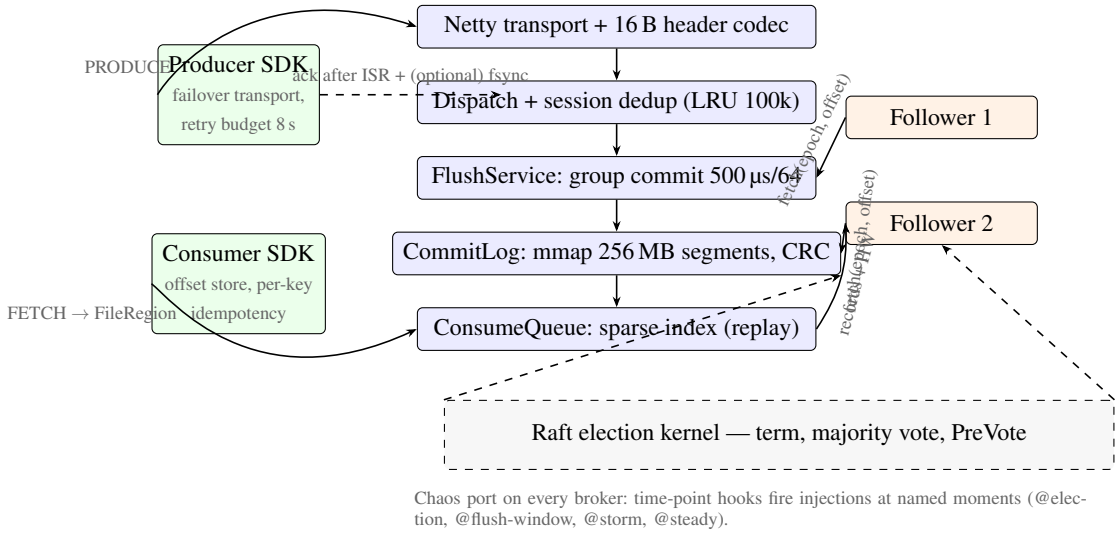


Figure 1: **mini-MQ architecture.** Control plane (dashed): a Raft-style kernel elects the leader; data plane (solid): followers pull fetch ranges with leader epochs, and consumers see only the high-watermark (HW) committed prefix. All topics share one sequentially appended CommitLog; a sparse ConsumeQueue index is rebuilt by replay. Every broker exposes a chaos port used by the fault matrix.

caliber) and cold consumption with the page cache retained reaches 760,678 msg/s.

3.2 Wire protocol

A 16-byte fixed big-endian header precedes each protobuf body: *magic* (rejects garbage before any parse), *version*, *type*, *request-id* (pairs asynchronous responses), and *body-length* (the framing unit and the DoS bound—bodies above the frame budget are trimmed, not buffered). The replication path originally missed the trim; the resulting silent drops were fixed and fenced with a regression test—one of several cases where the acceptance harness caught a divergence between two paths that should have shared a mechanism.

3.3 Replication data plane

Followers *pull* byte ranges (`fetch`) and the leader arbitrates them with **leader epochs**: on first contact after a leadership change, a follower's epoch is corrected and any uncommitted tail is truncated—which is exactly how a resurrected stale leader is stopped (cell 09, §4). Consumers observe only the **high watermark**, the committed prefix acknowledged by the full ISR. With `min.insync.replicas`=2, an ISR that shrinks below two makes the leader *fail loudly* (`ISR_TOO_SMALL`) instead of degrading silently; the resulting unavailability window is reported separately from RTO (§4).

3.4 Election control plane

Elections use a Raft-style kernel: monotonic terms, majority voting, and PreVote [16]—a candidate must first confirm (without incrementing its term) that a majority would vote for it, which prevents a partitioned node from disrupting a healthy leader on rejoin. Keeping election in a Raft core while the data plane stays Kafka-style fetch mirrors the KRaft split [18]: total-order consensus is used where a single decision lives (leadership), and per-partition efficiency is kept where throughput lives (replication).

3.5 Reliability services

Three tiers of persistence—**SYNC** (ack after `fsync`), **ASYNC** (ack before a background `fsync`; crash window declared), and **OS** (page cache only; power-loss window declared)—make the durability/throughput trade-off an explicit, testable configuration, in the spirit of `innodb_flush_log_at_trx_commit`. Producer retries deduplicate through a leader-session-scoped **LRU window** (100k entries) keyed by `seq_key`: same-key replays return `DUPLICATE` with the original offset. A client retry budget (8 s) and a dead-letter queue bound poison-message loops. Cross-session exactly-once is explicitly out of scope; consumer-side idempotency by `seq_key` is the declared contract.

4 Chaos-Verified Reliability

4.1 Deterministic fault injection

A fault that fires at a random time exercises one moment of the state space; a fault that fires at a *named moment* is a repeatable experiment. mini-MQ's injection toolkit has three parts: (1) a **ChaosRunner** that boots the cluster, drives load, and executes the cell script; (2) a **control side-channel** into a per-broker *chaos port*; and (3) a **PartitionShield** TCP proxy that drops traffic directionally for bidirectional and one-way partitions. Injections fire at atomic-counter *time-point hooks* instrumented into the broker: `@election` (a leader is being elected), `@flush-window` (a group-commit window is open), `@storm` (write load is at full speed), and `@steady` (steady state). The same command reruns the same cell at the same moment. We found no off-the-shelf match for this granularity: chaos frameworks such as Chaos Mesh [1] target container orchestration, while our clusters are three bare JVM processes—so the matrix would pay the cost of a toolchain without gaining its reach [14].

4.2 The 12-cell matrix

Table 1 shows the matrix. Each cell ends with hard assertions: consumed must equal acked (loss), per-producer sequences must be intact (order), duplicates are counted and, where expected by declared retry semantics, explained; fabrications (records that were never sent) must be zero. All twelve cells pass with **RPO** $= 0$: no message acknowledged under `acks=all` was ever lost.

4.3 RTO, and honesty about unavailability

For *recoverable* faults, the worst client retry chain we measured was **2.9 s**—against an 8 s budget composed of detection ≤ 1.5 s, election ≤ 0.5 s, and client redirection ~ 1 s, with the protocol segment alone measured at 1,152 ms in earlier milestone runs. But two cells produce *unavailability windows that are not RTO*, and folding them in would flatter us: after an ISR shrinks below `min.insync.replicas`, the leader rejects `acks=all` writes for 18.3–23.7 s (across runs) until the follower rejoins; a minority-partitioned leader cannot write at all for 4.4–4.7 s. Both are reported as what they are: windows in which mini-MQ chooses to be *loudly unavailable rather than silently wrong*.

4.4 The bugs the matrix caught

A matrix's credibility is its ability to go red. Two real bugs surfaced before 12/12 green:

Stale-HW reads through a minority leader. The first bidirectional-partition run exposed a leader isolated with a minority that kept serving FETCHes from *its* view of the high watermark— a view that no majority had ratified. The fix was three-part: a minority leader answers FETCH with `ISR_TOO_SMALL` instead of data; leader change cancels in-flight stale follower FETCHes; and a client whose transport hint goes stale drops the hint and rescans the bootstrap list.

ISR accounting race at full write speed. Under sustained max-rate appends, the catch-up check compared followers against a *live* LEO that advanced with every check, so a healthy follower could never "arrive" and was falsely evicted—5,308 failed writes out of 60,000 in one pre-fix run. The fix freezes the catch-up target at fetch-arrival time (a snapshot, so the bar cannot move mid-judgment), adds the frame-budget trim the replication path was missing, and teaches `appendReplicated` to roll the segment when the expected offset equals the next segment base. After the fix, an 18-round full-rate run shows zero write failures and evictions only from startup races; the regression test `healthyFollowerKeepsPaceUnderSustainedAppends` pins it.

5 Evaluation

5.1 Methodology

Environment. All cluster and comparison runs execute in WSL2 Ubuntu on one workstation (Intel Core i9-14900K, 32 logical cores as seen by the guest; OpenJDK 17.0.20.1; ext4 on the WSL2 virtual disk). The single-broker SYNC sweep in Figure 3 was taken on the Windows host of the same machine, as labeled. Kafka is 4.1.2 (KRaft mode). **Workloads.** 1 KiB messages unless stated; produce latency is measured to ack; tail-consume latency is message age at the consumer. **Rigor.** Every run reconciles: the harness asserts consumed $=$ produced with no duplicates, disorder, or fabricated records, or it fails. Single-round variance in loopback microbenchmarks measured up to 75%, so every headline number is the median of three rounds (per-round samples shown as scatter in the figures). All figures in this paper are *generated* from committed JSONL evidence by `paper/make-figures.py`; no number is hand-copied. The full regression suite is 209 tests, 0 failures, including the 12-cell chaos matrix.

5.2 Group commit: pricing the durability wall

Forced durability is the classic fsync wall: with one `fsync` per record, mini-MQ measures **83 msg/s**, against a no-`fsync` ceiling of 3.159 M msg/s on the same engine—a 38,055$\times$ spread. Group commit closes almost all of it. Figure 2(a) shows the counter-intuitive curve the mechanism predicts and the experiment reproduces: SYNC-tier throughput *increases* with concurrency, because concurrency is precisely what the 500 µs

#	Fault × moment (injection hook)	RPO	Visible dups	Per-cell assertion (all passed)
01	`kill -9` leader, mid write-storm (@storm)	0	0	acked 50 = consumed 50, sequences intact
02	`kill -9` leader, flush window open (@flush-window)	0	0	acked 46 = consumed 46; SYNC semantics honored
03	`kill -9` leader, mid election (@election)	0	0	acked 46 = consumed 46
04	`kill -9` follower, mid write-storm (@storm)	0	0	acked 51 = consumed 51; ISR shrank, service continued
05	`kill -9` follower, flush window (@flush-window)	0	0	acked 54 = consumed 54
06	Bidirectional partition, mid storm (@storm)	0	1	acked 51, consumed 52; the 1 duplicate is declared retry semantics
07	Bidirectional partition, mid election (@election)	0	0	acked 51 = consumed 52 (reconciled by epoch arbitration)
08	One-way partition, steady state (@steady)	0	0	acked 50 = consumed 50; isolated old leader fenced
09	Stale-leader revival (old leader rejoins serving)	0	0	acked 65 = consumed 65; term arbitration ends it on first contact
10	ISR shrink below `min.insync.replicas`=2	0	0	loud `ISR_TOO_SMALL`, no silent degradation
11	Minority partition	0	0	unavailable rather than wrong (window reported, §4.3)
12	Retry storm against dedup window	—	—	same `seq_key` replays return DUPLICATE + original offset

Table 1: **The 12-cell chaos matrix** (three-node cluster, real `SIGKILL`s, TCP-level partitions, deterministic time-point hooks). RPO is hard-asserted per cell. Cell 06's single duplicate is at-least-once retry semantics made visible rather than hidden; same-session duplicates collapse in the dedup window (cell 12), and cross-leader duplicates are the consumer's idempotency contract. The whole matrix reruns with `mvn -pl client -am test -Dtest=ReliabilityMatrixTest` (∼248 s) and is part of the 209-test regression suite.

Flush tier	Ack semantics	Acked	Lost
SYNC	after `fsync`	65,336	**0**
ASYNC	before background force	90,488	**0**
OS	page cache only	60,484	**0**

Table 2: **Crash semantics, measured with real `SIGKILL`s.** A loaded single broker is killed at a random moment after 10k acks, restarted through CRC recovery, and reconciled. SYNC's zero is the tier's *contract*; ASYNC's and OS's zero-loss results are honest windows-that-were-not-hit, not proofs: ASYNC declares a crash exposure window (unforced tail), OS declares a power-loss window. The harness distinguishes the two statements.

flush window converts into batch size—records per `fsync` rise from 241 to 909 as producers go from 1 to 8, and by 8 producers the full cost of forcing is **11%** of the no-force tier. This is the same amortization MySQL's `ordered_commit()` performs [3, 9], with two independent knobs—*how long* to accumulate (linger) and *how much* (count)—mirroring the delay/count pair of the binlog. Figure 2(b) is the 5×4 scan that chose the shipped defaults: the surface is a plateau (∼208–213k msg/s) with exactly one collapsing corner (a window too small to aggregate, 18,691 msg/s) and one degradation ridge (a window longer than the SDK's own batching period, ∼112k). The default sits on the plateau at the P99 knee's left edge; the choice, and the scan, are part of the artifact.

Table 2 prices the tiers against reality: a broker `SIGKILL`ed mid-load, restarted, and reconciled. One subtlety the harness enforces: an ASYNC or OS run that loses nothing must be reported as *window not hit*, not as evidence the window does not exist. (Early in development, the shared flush counter also had a race between concurrent appends and the in-flight force; it was replaced by a byte-monotonic `flushedOffset` with a dedicated unit test.)

5.3 Single broker

Figure 3 shows the full protocol stack (client, Netty, framing, dispatch, CommitLog, index) at the SYNC tier: a throughput plateau of 157–170k msg/s whose P99 stays flat at ∼110 ms, i.e., the plateau is the honest ceiling of this stack on this host, not a queueing collapse. On the consume side, zero-copy delivery uses `FileRegion`/`sendfile` [2] because the produce path *cannot* be zero-copy: it must write, index, and CRC in user space. We report an inconvenient A/B honestly: in WSL2's virtualized I/O stack, the sendfile path measured *11.8% slower* than inline assembly of responses (three-round medians). The decision record keeps the mechanism (real NICs amortize the syscall tax) while declaring the loopback result as-is.

5.4 Three-node cluster vs. same-host Kafka

The headline comparison aligns seven dimensions (message size, durability semantics, replication factor, `min.insync.replicas`, compression, topology, workload) and *declares* the two it cannot erase: mini-MQ's SDK batches at linger 500 µs / 64 records vs. Kafka's 16 KB / 5 ms, and rounds are 60k vs. 600k records (10× less JIT amortization per round for mini-MQ). Figure 4 shows the result: 38,001 vs. 74,776 msg/s (**51%**), P99 1,555 vs. 943 ms (**1.65**×), P50 at parity. The flat 1-to-16-producer plateau localizes the bottleneck inside the replication loop before any profiling is done.

Table 3 is the paper's core honesty artifact: the gap decomposed into four profiled buckets, one of which (storage) the

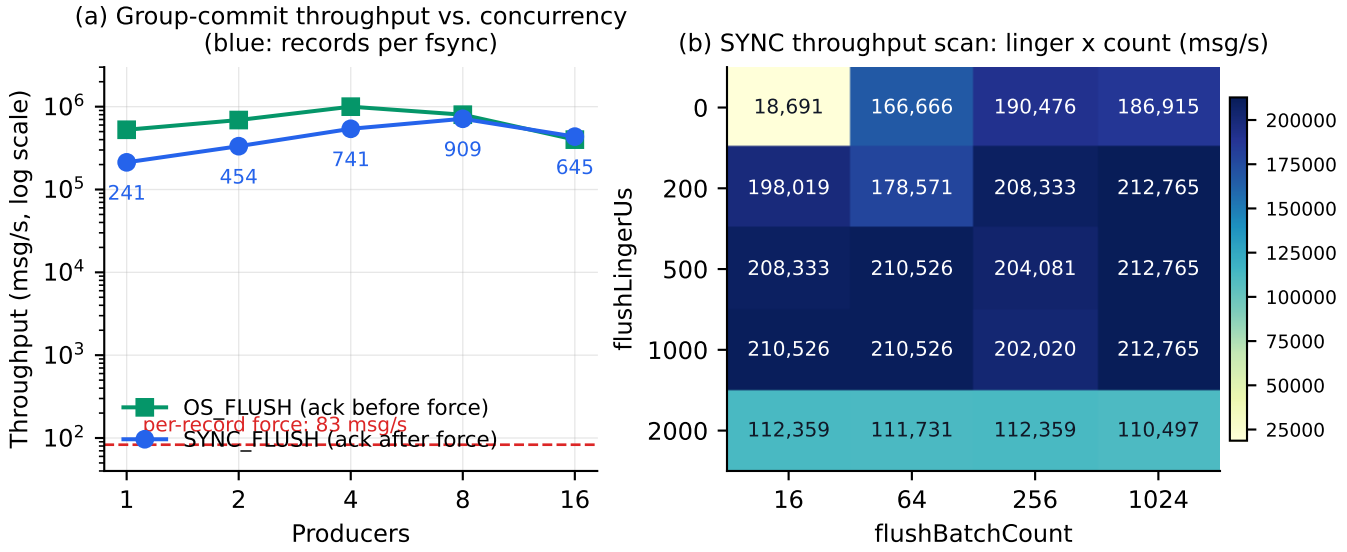


Figure 2: **Group commit.** (a) SYNC-tier throughput *rises* with producer concurrency (212,765 → 714,285 msg/s) because the flush thread's 500 µs window aggregates more records per `fsync` (blue labels: records per force, 241 → 909); the per-record-`fsync` baseline is 83 msg/s, 38,055× below the no-force ceiling. At 8 producers the entire cost of forcing is 11% (714,285 vs. 800,000 OS-FLUSH). At 16 producers both tiers fall back—the bottleneck moves to the protocol layer, not the disk. (b) The 5×4 parameter scan that motivated the shipped default (linger 500 µs / count 64): throughput plateaus across the scan, collapsing only in the pathological corner (18,691 msg/s at linger 0 / count 16, i.e., near-per-record forcing) and degrading when the window exceeds the SDK's batching period (linger 2,000 µs). Data: M2 `BenchV2` runs archived in `docs/bench-v2.md`.

evidence explicitly *exculpates*, and one caution note refusing to over-read its strongest number (the 91.6% G1 share is measured at the SYNC single-broker full-load caliber; treating it as the RF=3 gap's quantitative split would be dishonest, so it stays evidence of a scaling wall, not a decomposition).

5.5 Profiling-driven optimization

All three optimization stories follow the same discipline: profile first (JFR [8] and async-profiler [17] flame graphs are committed), change the minimum, re-measure medians of three.

The send-path critical section. The lock profile showed the producer's send critical section wrapping state it did not need to share. Shrinking it to the single shared variable (Figure 5) moved lock samples from 2.4% to 0.3% of the profile and throughput by +3.4%.

Stack-rental object pools. JFR allocation sampling pinned the hot allocations to encode buffers and CRC32 state. The pools are *stack-rental*: a buffer is borrowed from a `ThreadLocal`, is bound to the borrowing call stack, and has no cross-thread escape and no explicit recycle protocol—use-after-recycle is unrepresentable in the types. Sampled allocations fell 17.3% on the inline path (34.0% where zero-copy also applies), the hot frames vanished from the JFR top-25, and measured pool hit rates are 100% (17,920,418 of 17,920,450 over a 30-minute run).

The ISR race. Already detailed in §4.4: 5,308/60,000 failed writes → 0 after the frozen-target algorithm, with the regression test pinning the fix. We note the pattern across the three stories: each was *found* by an instrument (lock profile, allocation profile, chaos cell) rather than by code review, and each fix is fenced by a dedicated test.

5.6 Stability: what only a long run finds

A 30-minute rate-limited soak (10k msg/s, 8 producers, 512 B records, ∼10.1 GB across ∼38 segments, Netty leak detection at `paranoid`) reconciled **17,920,450 acked = 17,920,450 consumed** with per-record sequence verification: zero loss, zero duplicates, zero disorder, zero fabricated payloads. Throughput decayed +0.2% between the first and last five-minute windows (i.e., none), GC trended flat (22/min falling to 6/min), leak reports were zero.

The soak's unique catch: at minute 7, consumption halved, then collapsed, while GC spiked to 268 collections/min and CPU burned in `ConsumeQueue` dispatch. The cause was a `int`-multiplication overflow in segment lookup (`idx * segmentSize`) exactly at the 2 GiB boundary (segment 8's base $= 2^{31}$): the cursor silently reset and dispatch re-read the segment head in an infinite loop, re-CRCing the same 512 records ∼767× each. Every other test in the suite writes less than 2 GB—*only* the soak crossed the boundary. The fix (range comparison in `long`) is fenced by a regression

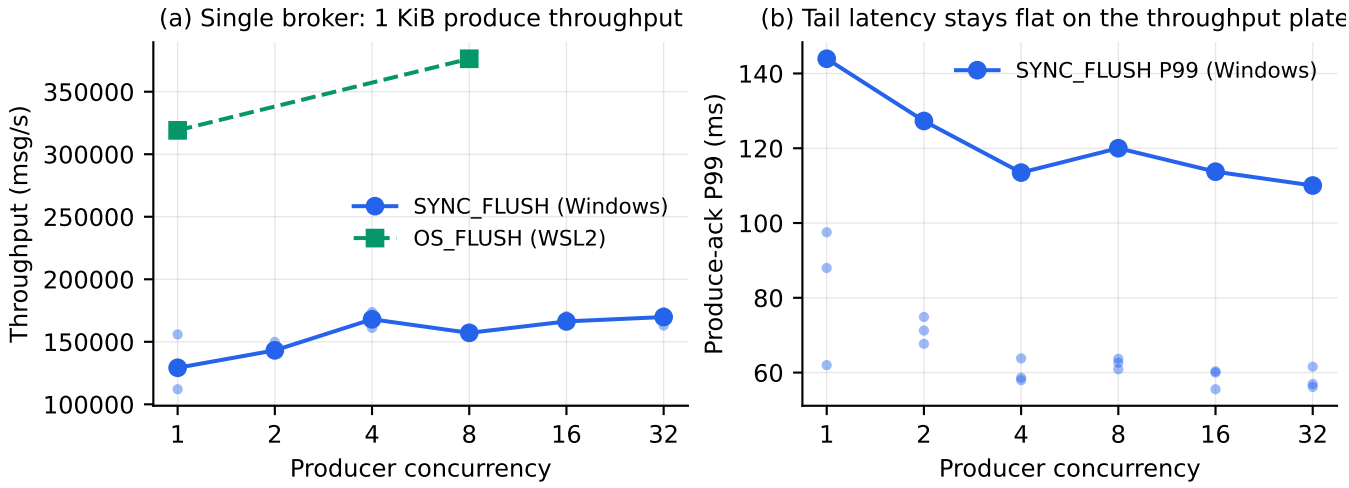


Figure 3: **Single-broker produce.** Throughput (a) and produce-ack P99 (b) across producer concurrency, full protocol stack, 1 KiB records, three rounds per point (dots) with medians (line). SYNC (Windows host) plateaus at 157–170k msg/s with P99 flat at ∼110 ms; OS-FLUSH (WSL2, shown for scale) reaches 376,350 msg/s. Every point reconciles consumed = produced. Data: `result/m5-20261017-verified/produce.jsonl`, `result/m5-20261017-rf3-wsl/minimq-wsl-produce-os.jsonl`.

test that hand-builds a sparse 3×1 GiB segment chain so the boundary is testable without writing 2 GB. The lesson generalizes: byte-offset arithmetic belongs in `long` everywhere, and long-horizon runs are the only tests that reach quantity-becomes-quality failure modes.

6 Limits, Declared

We state what mini-MQ deliberately is not, because a system's claims are only as good as its stated boundaries [15]:

- **No cross-session exactly-once.** Dedup is leader-session-scoped; a retry that lands on a different leader appends again. Final arbitration is consumer-side idempotency by `seq_key`.
- **One write stream per broker.** All topics share a single CommitLog, so write parallelism is bounded by one sequential file. The decision is recorded with its invalidation boundary: it flips the day single-file sequential writes become the bottleneck.
- **No consumer-group rebalance** (subscriptions are static) **and no security layer** (no TLS, no authentication, no ACLs).
- **Scale evidence is three nodes on one host (loopback).** No claim is made about 30 nodes, cross-DC replication, or cloud topologies.
- **Benchmark numbers are Linux-side (WSL2).** The Windows host runs the tests, but NTFS-native mmap semantics are not a maintained benchmark caliber; and no `docker-compose` is shipped, because the development machine's Docker daemon could not validate one—an untested one-click file would break the ten-minute promise it makes.
- **A declared share of the Kafka gap is unattributed** (Table 3), and honest attribution is worth more than a fully-explained fiction.

7 Related Work

Production message queues. Kafka [5, 12] established the partitioned-log model with pull replication, leader epochs, and ISR semantics; RocketMQ [7] the single CommitLog with replayed consume queues; Pulsar [6] segmented ledgers and compute-storage separation. mini-MQ borrows one gene from each and, uniquely among the three, is small enough to read in an afternoon. **Consensus and control planes.** Raft's understandability argument [16] motivated our election kernel; KRaft [18] validated separating a Raft-style metadata plane from fetch-based data replication, a split mini-MQ shares. **Educational systems.** xv6 [4] plays the same role for operating systems that mini-MQ aims to play for messaging: real mechanisms, executable size. **Group commit** is classical transaction-processing wisdom [9], productionized in MySQL's `ordered_commit()` [3]. **Chaos engineering.** The Simian Army [14] and the chaos discipline [1] injected faults in production; mini-MQ contributes *deterministic, moment-named* injection at bare-JVM scale, where container-era tooling does not reach.

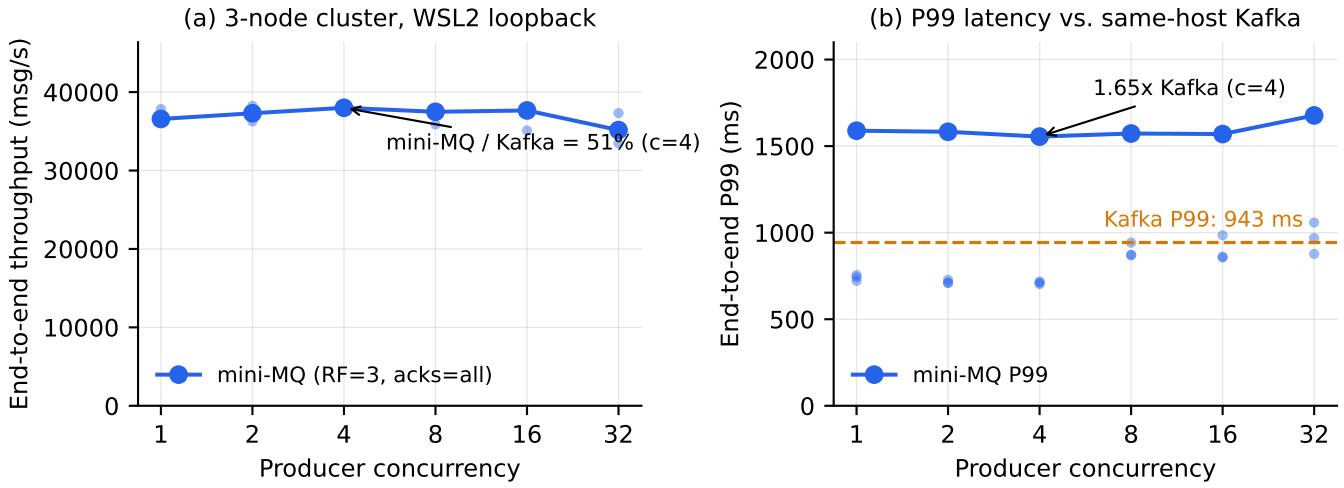


Figure 4: **End-to-end, RF=3, `acks=all`, vs. Kafka 4.1.2 on the same host.** mini-MQ holds a 35–38k msg/s plateau from 1 to 16 producers (32 dips: the bottleneck is inside the replication loop, not client concurrency); Kafka's median is 74,776 msg/s. P99: mini-MQ 1,555 ms vs. Kafka 943 ms (1.65×), while P50 is at parity (711 vs. 725 ms, 0.98×)—the gap is tail-shaped. Every point reconciles; medians of three rounds with per-round scatter. Data: `result/m5-20261017-rf3-wsl/`.

8 Conclusion

mini-MQ shows that the load-bearing mechanisms of industrial message queues—sequential-log storage with CRC recovery, two-parameter group commit, pull replication with epoch arbitration and watermarks, a Raft election kernel, deduplication and dead letters—can be built honestly at learning scale: with faults injected at named moments, with invariants hard-asserted per cell, with optimizations driven by committed flame graphs, and with the industry comparison reported as a loss (51%, 1.65×) whose anatomy is profiled bucket by bucket and whose unexplained remainder is declared rather than decorated. We believe the reusable contribution is the *caliber*: a recipe for what a small system must measure, assert, and admit in order for its green lights to mean something.

Availability

mini-MQ is open source under the MIT license:

`https://github.com/Xstly1014/mini-MQ`

One command builds it, boots a three-node cluster, reconciles a 10,000-message smoke run (asserting `lost=0 dup=0`), and tears it down:

```
bash bench/quickstart.sh
```

Every performance number in this paper regenerates from committed evidence files: `paper/make-figures.py` rebuilds all figures from the JSONL artifacts in `result/`, and the benchmark scripts in `bench/` rerun every experiment end to end. The 209-test regression suite, including the 12-cell chaos matrix, runs with `mvn clean test`. Eight architecture decision records (`docs/adr/`) document each major choice with its rejected alternatives.

References

[1] Ali Basiri, Niosha Behnam, Ruud de Rooij, Lorin Hochstein, Luke Kosewski, Justin Reynolds, and Casey Rosenthal. Chaos engineering. *IEEE Software*, 33(3):35–41, 2016.

[2] Sapan Bhatia. Efficient data transfer through zero copy. IBM developerWorks, 2004. https://developer.ibm.com/articles/j-zerocopy/.

[3] Oracle Corporation. MySQL server source code: `sql/binlog.cc`, `mysql_bin_log::ordered_commit`. https://github.com/mysql/mysql-server/blob/trunk/sql/binlog.cc, 2025.

[4] Russ Cox, Frans Kaashoek, M. and Robert T. Morris. xv6: A simple, unix-like teaching operating system. MIT PDOS, https://pdos.csail.mit.edu/6.828/2018/xv6.html, 2018.

[5] Apache Software Foundation. Apache Kafka documentation — design: Replication. https://kafka.apache.org/documentation/#replication, 2014.

[6] Apache Software Foundation. Apache Pulsar documentation — pulsar architecture. https://pulsar.apache.org/docs/, 2025.

Bucket	Evidence (leader-side profiles)	Verdict
① Replication advances per record	Each replicated record is one protobuf entry and one `appendReplicated` call; fetch round-trips are linear in record count. Kafka writes an entire produce request's records in one pass.	Structural: the flat c=1→16 curve (§5.4) is consistent with a per-record loop ceiling.
② Protocol encoding	Serialization frames (protobuf codec + MessageCodec) are 2.2% of leader CPU samples at SYNC full load.	Real but secondary; Kafka's fixed binary format amortizes framing over batches.
③ Storage path	CommitLog mmap append + ConsumeQueue dispatch are 2.2% of leader CPU samples.	Exculpated: both systems are page-cache sequential writes; the single CommitLog ≈ one Kafka partition log.
④ Thread & memory model	91.6% of leader CPU samples are G1/VM frames (6,177 of 6,742; e.g., `OtherRegionsTable::add_reference` 2,695, `G1HotCardCache::insert` 679) at the 176k msg/s SYNC full-load caliber.	Per-record allocations on the hot path (replication read buffers, protobuf arrays, message objects) make remembered-set maintenance the CPU majority; Kafka's batch-oriented path allocates near-zero per record. *Caution: this bucket is a scaling wall demonstrated at the SYNC full-load caliber; it is not a quantitative split of the 49% RF=3 gap.*
Unattributed	—	Declared, not hidden: same-host resource contention between the two 3-node clusters, the 10× round-length asymmetry, and two decades of end-to-end optimization in Kafka.

Table 3: **Four-bucket attribution of the gap to Kafka.** Each bucket carries profiled evidence; none is guessed. The share that cannot honestly be attributed is reported as unattributed.

[7] Apache Software Foundation. Apache RocketMQ documentation — architecture. https://rocketmq.apache.org/docs/, 2025.

[8] Markus Ghani et al. JEP 328: A Java flight recorder subsystem. OpenJDK Enhancement Proposal, https://openjdk.org/jeps/328, 2018.

[9] Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993.

[10] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[11] Jay Kreps. The log: What every software engineer should know about real-world data's usability. LinkedIn Engineering Blog, 2013. https://engineering.linkedin.com/distributed-systems/log-what-every-software-engineer-should-know-about-real-world-datas-usability.

[12] Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: A distributed messaging system for log processing. In *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB)*, 2011. https://netdb07.cs.stanford.edu/papers/netdb07-01.pdf.

[13] Linux man-pages project. mmap(2) — linux programmer's manual. https://man7.org/linux/man-pages/man2/mmap.2.html, 2024.

[14] Netflix. The Netflix simian army. Netflix Technology Blog, 2011. https://netflixtechblog.com/the-netflix-simian-army-16e57fbab116.

[15] Michael Nygard. Documenting architecture decisions. Relevance Blog (self-published), 2011. https://web.archive.org/web/20130523222945/http://thinkrelevance.com/blog/2011/11/15/documenting-architecture-decisions.

[16] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm (extended version). In *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pages 305–319, 2014. https://www.usenix.org/system/files/conference/atc14/atc14-paper-ongaro.pdf.

[17] Andrei Pangin. async-profiler. https://github.com/async-profiler/async-profiler, 2025.

[18] Jun Rao et al. KIP-500: Disentangling controller from ZooKeeper. Apache Kafka Improvement Proposal, Apache Software Foundation, 2020. https://cwiki.apache.org/confluence/display/KAFKA/KIP-500%3A+Disentangling+Controller+from+ZooKeeper.

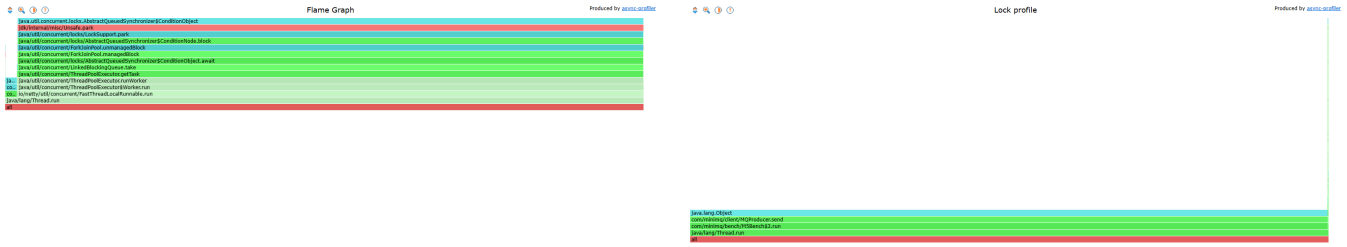


Figure 5: **Lock profiles, before and after shrinking the producer's send critical section** (8-producer SYNC full load). Left: before—the critical section wrapped state it did not need to share, and lock samples are 2.4% of the profile. Right: after—the critical section holds only the shared variable; lock samples fall to 0.3% and median throughput rises 171,865 → 177,691 msg/s (+3.4%, medians of three rounds). Caveat declared: the two profiles come from different toolchains (JFR before, async-profiler after), so the comparison is of normalized shares, not raw sample counts. Evidence: `result/flame/m5-lock-produce8c.html`, `result/m5-20261017-rf3-wsl/m5-wsl-lock-produce8c-after.html`.